COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S4
DATE PERFORMED:	07/22/26	**DATE SUBMITTED:**	07/22/26
NAME:	Francis Elijah J. Formalejo	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 3.1 Creating a Singly Linked List

1. PROCEDURE OUTPUT

What procedures did you perform?

Main:

```cpp
#include <iostream>
#include "singly_ll.h"

int main()
{
    SingleList<char>* head = new SingleList<char>();
    SingleList<char>* second = new SingleList<char>();
    SingleList<char>* third = new SingleList<char>();
    SingleList<char>* fourth = new SingleList<char>();
    SingleList<char>* fifth = new SingleList<char>();

    head->data = 'P';
    second->data = 'E';
    third->data = 'D';
    fourth->data = 'S';
    fifth->data = 'S';

    head->next = second;
    second->next = third;
    third->next = fourth;
    fourth->next = fifth;
    fifth->next = nullptr;

    std::cout << "Testing of Traversal:\n";
    ListTraversal(head);

    std::cout << "\nTesting of sll_insert_head:\n";
    sll_insert_head('Q', &head);
    ListTraversal(head);

    std::cout << "\nTesting of sll_general_insert:\n";
    sll_general_insert('X', head->next->next);
```

```
    std::cout << "\nTesting of deleting the node:\n";
    sll_Delete_Node('U', &head);
    ListTraversal(head);

    sll_Delete_Node('Q', &head);
    ListTraversal(head);

    sll_Delete_Node('X', &head);
    ListTraversal(head);

    std::cout << "\nDeleting all nodes:\n";
    sll_Deleting_List(&head);
    ListTraversal(head);

    return 0;
}
```

Singly:

```
#ifndef SINGLY_LL_H
#define SINGLY_LL_H

#include <iostream>

template <typename T>
class SingleList
{
public:
    T data;
    SingleList<T>* next = nullptr;
};

template <typename T>
void ListTraversal(SingleList<T>* head)
{
    if (head == nullptr)
    {
```

```
    {
        std::cout << "The list is empty." << std::endl;
        return;
    }

    while (head != nullptr)
    {
        std::cout << head->data;

        if (head->next != nullptr)
            std::cout << " -> ";

        head = head->next;
    }

    std::cout << std::endl;
}

template <typename T>
void sll_insert_head(T newData, SingleList<T>** currentHead)
{
    SingleList<T>* newNode = new SingleList<T>;

    newNode->data = newData;
    newNode->next = *currentHead;

    *currentHead = newNode;
}

template <typename T>
void sll_general_insert(T newData, SingleList<T>* prevNode)
{
    if (prevNode == nullptr)
    {
        std::cout << "Previous node cannot be nullptr." << std::endl;
        return;
```

```
        return;
    }

    SingleList<T>* newNode = new SingleList<T>;

    newNode->data = newData;
    newNode->next = prevNode->next;
    prevNode->next = newNode;
}

template <typename T>
void sll_insert_end(T newData, SingleList<T>** head)
{
    SingleList<T>* newNode = new SingleList<T>;

    newNode->data = newData;
    newNode->next = nullptr;

    if (*head == nullptr)
    {
        *head = newNode;
        return;
    }

    SingleList<T>* currentNode = *head;

    while (currentNode->next != nullptr)
    {
        currentNode = currentNode->next;
    }

    currentNode->next = newNode;
}

template <typename T>
```

```cpp
template <typename T>
void sll_Delete_Node(T findData, SingleList<T>** head)
{
    if (*head == nullptr)
        return;

    SingleList<T>* currNode = *head;
    SingleList<T>* prevNode = nullptr;

    while (currNode != nullptr && currNode->data != findData)
    {
        prevNode = currNode;
        currNode = currNode->next;
    }

    if (currNode == nullptr)
    {
        std::cout << "The data is not found." << std::endl;
        return;
    }

    if (prevNode == nullptr)
    {
        *head = currNode->next;
    }
    else
    {
        prevNode->next = currNode->next;
    }

    delete currNode;
}

template <typename T>
void sll_Deleting_List(SingleList<T>** head)
{
```

```
{
    SingleList<T>* current = *head;

    while (current != nullptr)
    {
        SingleList<T>* temp = current;
        current = current->next;
        delete temp;
    }

    *head = nullptr;
}

#endif
```

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

COMPUTER ENGINEERING

QUEZON CITY

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.

Conclusion:

Overall, we tried the singly linked list and learned how memory allocation works with pointers. We linked nodes to process some datas and listtraversal for displaying list in elements in order. Silinserthead for inserting nodes at the start of the list and sllgeneralinsert for inserting a node after a target node, and last the slldeletenode to remove the target node.